

1. ÖZYİNELEMELİ (RECURSİVE) OLMAYAN ALGORİTMALARIN ANALİZİ

Örnek 1: n sayıdan oluşan bir listedeki en büyük elemanın değerini bulma problemini ele alalım. Basitlik açısından, listenin bir dizi olarak uygulandığını varsayalım.

Algoritma – MaxElement($A[0..n - 1]$)

maxval $\leftarrow A[0]$

for $i \leftarrow 1$ to $n - 1$ do

 if $A[i] > \text{maxval}$

 maxval $\leftarrow A[i]$

return maxval

Algoritma:

1. İlk elemanı ($A[0]$) en büyük değer olarak ata.
2. Dizinin geri kalan elemanlarını ($A[1]$ 'den $A[n-1]$ 'e kadar) sırayla kontrol et.
3. Daha büyük bir değer bulunursa, onu güncelle.
4. En büyük değeri döndür.

Bu algoritma **$O(n)$** zaman karmaşıklığına sahiptir, çünkü tüm elemanlar bir kez kontrol edilir.

Bu metin, algoritmanın temel işlemlerini analiz ediyor.

- Girdi boyutu **n** , dizideki eleman sayısı ile belirlenir.
- En sık yapılan işlemler, **for döngüsü** içinde gerçekleşir.
- Döngüde iki işlem vardır:
 1. **Karşılaştırma:** $A[i] > \text{maxval}$
 2. **Atama:** $\text{maxval} \leftarrow A[i]$

- Temel işlem olarak **karşılaştırma** seçilmelidir çünkü döngünün her tekrarında gerçekleşir, ancak atama yalnızca bazen yapılır.
- **Karşılaştırma sayısı** tüm girişler için aynıdır (n-1 kez yapılır), bu nedenle **en iyi, ortalama ve en kötü durumlar arasında bir fark yoktur.**

Karşılaştırma işleminin işlem sayısına bakalım

C(n), algoritmadaki karşılaştırma sayısını ifade eder.

Her döngü turunda bir karşılaştırma yapılır ve döngü 1'den (n-1)'e kadar çalışır.

Toplam karşılaştırma sayısı:

$$C(n) = \sum_{i=1}^{n-1} 1$$

Bu toplam, **(n-1) kere 1'in toplanmasıyla** elde edilir: $C(n) = n - 1$

Büyük-O gösterimiyle karmaşıklık: $C(n) \in \Theta(n)$

Sonuç olarak, **algoritmanın karşılaştırma işlemleri doğrusal zaman karmaşıklığına sahiptir .**

Yinelemeli olmayan algoritmaların zaman karmaşıklığını analiz etme sürecine adım adım bakalım.

Girdi büyüklüğünü belirle: Algoritmanın çalışmasını etkileyen parametreleri seç.

1. **Temel işlemi tespit et:** Genellikle en iç döngüde yer alan en sık yapılan işlemdir.
2. **Temel işlemin kaç kez çalıştığını belirle:**
 - Sadece giriş boyutuna mı bağlı?
 - En kötü, ortalama ve en iyi durumları ayırmak gerekir mi?
3. **Bir toplam formülü oluştur:** Temel işlemin tekrar sayısını matematiksel olarak ifade et.
4. **Bu toplamı hesapla veya büyüme düzenini belirle:**

- Kapalı bir formül bulun veya **büyük-O gösterimiyle** büyüme oranını tahmin et.

Ek olarak, **iki temel toplam (sum) manipülasyon kuralı** verilmiştir:

- **R1:** Sabit çarpan toplam dışına alınabilir.
- **R2:** Toplam içindeki terimler ayrı ayrı toplanabilir veya çıkarılabilir.

R1:

$$\sum_{i=l}^u c a_i = c \sum_{i=l}^u a_i$$

R2:

$$\sum_{i=l}^u (a_i \pm b_i) = \sum_{i=l}^u a_i \pm \sum_{i=l}^u b_i$$

S1:

$$\sum_{i=l}^u 1 = u - l + 1$$

- **Sabit bir değerin toplamı** hesaplanıyor.
- Örneğin, **Örnek 1'de kullanılan** $\sum_{i=1}^{n-1} 1 = n - 1$ bu formülün özel bir durumudur.

S2:

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2)$$

- **İlk n doğal sayının toplamı** veriliyor.
- Büyüme oranı $\Theta(n^2)$ olduğu için **karesel büyüme gösteren algoritmalarda** sıkça kullanılır.

Bu formüller, **algoritmaların çalışma süresini tahmin ederken** ve **büyük-O analizleri yaparken** önemli bir temel oluşturur.

Örnek 2:

Algoritma – UniqueElements(A[0..n – 1])

```
for i ← 0 to n – 2 do
    for j ← i + 1 to n – 1 do
        if A[i] = A[j] return false
return true
```

Bu örnek, **eleman benzersizliği (element uniqueness) problemini** ele almaktadır. Amaç, bir dizideki tüm elemanların **farklı olup olmadığını** kontrol etmektir.

Algoritma Açıklaması:

- **İç içe iki döngü kullanılır:**
 - **Dış döngü (ii)** her elemanı seçer.
 - **İç döngü (jj)** kalan elemanlarla karşılaştırma yapar.
- Eğer **iki eleman eşitse, "false" döndürülür** (dizi benzersiz değildir).
- Eğer hiç eşleşme bulunmazsa, **"true" döndürülür** (tüm elemanlar farklıdır).

Zaman Karmaşıklığı:

- Dış döngü **n-1n-1** kez, iç döngü ortalama **n/2** kez çalışır.
- **Toplam karşılaştırma sayısı:**

$$\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \frac{n(n-1)}{2} \in \Theta(n^2)$$

- **Karmaşıklık: $O(n^2)$** (karesel zaman karmaşıklığı).

Alternatif Yaklaşımlar:

- **Sıralama + Tek Geçiş:**

- Diziyi sıralayıp, yan yana olan elemanları kontrol edersek $O(n \log n)$ sürede çalışabilir.

- **Hash Set Kullanımı:**

- Bir **hash set** ile elemanları kontrol edersek $O(n)$ sürede çözebiliriz.

Bu algoritma küçük diziler için kabul edilebilir olsa da, büyük dizilerde **hash tabanlı yaklaşımlar** veya **sıralama tabanlı optimizasyonlar** daha iyi performans sağlar.

Algoritmanın en kötü durum analizine bakalım. Girdi boyutu yine n olup, temel işlem iç içe geçmiş döngüde gerçekleştirilen eleman karşılaştırmasıdır. Karşılaştırma sayısı yalnızca n 'ye değil, aynı zamanda dizide aynı elemanların olup olmamasına ve bu elemanların konumlarına da bağlıdır. Analiz, en kötü durum senaryosu üzerinde yoğunlaşmaktadır. En kötü durumda, dizideki tüm elemanlar farklıysa veya yalnızca son iki eleman aynıysa, algoritma tüm karşılaştırmaları yapmak zorunda kalır. Böyle bir durumda iç döngü, her i değeri için j değişkenini $i + 1$ ile $n - 1$ arasında değiştirerek tam olarak çalışır ve bu işlem, dış döngüde i değişkeninin 0 'dan $n-2$ 'ye kadar değişmesiyle tekrar edilir. Sonuç olarak, algoritmanın en kötü durum zaman karmaşıklığı $O(n^2)$ olur.

$$\begin{aligned}
C_{\text{worst}}(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) \\
&= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} \\
&= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2).
\end{aligned}$$

Toplam $\sum_{i=0}^{n-2} (n-1-i)$ aşağıdaki şekilde daha hızlı hesaplanabilir:

$$\sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \dots + 1 = \frac{(n-1)n}{2}$$

Bu metin, en kötü durum analizinde kullanılan **toplam formüllerine (S2)** yapılan referansı açıklıyor. En kötü durumda, algoritma **tüm olası $n(n - 1)/2$ farklı eleman çiftini** karşılaştırmak zorundadır. Bu sonuç **beklenen bir durumdur**, çünkü iç içe geçmiş döngü her eleman çiftini karşılaştıracak şekilde tasarlanmıştır. Bu nedenle, **toplam karşılaştırma sayısı** doğrudan $O(n^2)$ zaman karmaşıklığını doğrular.

Örnek 3:

ALGORITHM MatrixMultiplication(A[0..n - 1, 0..n - 1], B[0..n - 1, 0..n - 1])

for i $\leftarrow$ 0 to n - 1 do

 for j $\leftarrow$ 0 to n - 1 do

 C[i, j] $\leftarrow$ 0.0

 for k $\leftarrow$ 0 to n - 1 do

 C[i, j] $\leftarrow$ C[i, j] + A[i, k] * B[k, j]

return C

Bu örnek, **matris çarpımının zaman karmaşıklığını** analiz etmektedir.

$$\begin{array}{c}
 \text{A} \qquad \qquad \qquad \text{B} \qquad \qquad \qquad \text{C} \\
 \text{row } i \left[\begin{array}{|c|c|c|c|c|} \hline \square & \square & \square & \square & \square \\ \hline \end{array} \right] * \left[\begin{array}{|c|} \hline \square \\ \square \\ \square \\ \square \\ \square \\ \hline \end{array} \right] = \left[\begin{array}{|c|} \hline C[i,j] \\ \hline \end{array} \right] \\
 \qquad \qquad \qquad \text{col. } j
 \end{array}$$

İki $n \times n$ boyutunda matris A ve B verildiğinde, çarpım sonucu $C = A \times B$ matrisi yine $n \times n$ boyutunda olacaktır. Matris çarpımı, **A'nın satırları ile B'nin sütunlarının skaler çarpımları** olarak hesaplanır.

Her bir $C[i, j]$ elemanı, **A'nın i. satırındaki elemanlarla B'nin j. sütunundaki elemanların çarpılıp toplanmasıyla** elde edilir. Bu işlem için:

- Her $C[i, j]$ elemanı için n çarpma ve toplama yapılır.
- C matrisi $n \times n$ olduğu için toplamda n^2 eleman hesaplanır.
- Her eleman n işlem gerektirdiğinden, toplam işlem sayısı:

$$n^2 \times n = n^3$$

- Sonuç olarak matris çarpma algoritmasının **zaman karmaşıklığı** $O(n^3)$ 'tür.

Matris çarpımının toplam çarpma işlemi sayısını analiz edebiliriz.

Matris boyutu n olarak tanımlanır. İç içe geçmiş döngüde **iki temel işlem** (çarpma ve toplama) bulunmaktadır, ancak bunlar her tekrar için aynı sayıda yapıldığından yalnızca **çarpma işlemleri sayılmaktadır**.

Çarpma işlemleri en iç döngüde gerçekleştirilir ve **kk** değişkeni 0 'dan $n - 1$ 'e kadar çalışır. Her $C[i, j]$ elemanı için **nn çarpma yapılır**.

Toplam çarpma sayısı üçlü toplam (triple sum) ile ifade edilir:

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

Bu toplam, her i ve j için n çarpma yapıldığını ve toplamda n^3 işlem gerektiğini gösterir.

Matris çarpımının zaman karmaşıklığı $O(n^3)$ olarak belirlenir.

İç içe geçmiş toplamalar, **formül (S1) ve kural (R1)** kullanılarak sadeleştirilmiştir. **En içteki toplamın** değeri n olduğundan, toplam çarpma sayısı şu şekilde hesaplanır:

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3$$

Bu hesaplama, her bir $C[i,j]$ elemanı için n çarpma işlemi yapıldığını ve toplam n^2 eleman olduğunu gösterir. Sonuç olarak, toplam çarpma işlemi sayısı n^3 olarak bulunur.

Bu analiz, **algoritmanın zaman karmaşıklığının $O(n^3)$ olduğunu gösterir.** Matris çarpımındaki her elemanın, **birinci matrisin bir satırının ve ikinci matrisin bir sütununun skaler çarpımı** olarak hesaplandığı ve bunun n çarpma gerektirdiği belirtilmiştir. Sonuçta, n^2 eleman için toplam n^3 çarpma yapılmaktadır.

Algoritmanın çalışma süresi, **makinedeki tek bir çarpma işleminin süresi (c_m) ile toplam çarpma sayısının ($M(n)$) çarpımı** ile yaklaşık olarak hesaplanabilir:

$$T(n) \approx c_m M(n) = c_m n^3$$

Ancak daha doğru bir tahmin yapmak için **toplama işlemleri de hesaba katılmalıdır.** Toplama işlemleri için de benzer bir katsayı (c_a) tanımlanarak:

$$T(n) \approx c_m M(n) + c_a A(n) = c_m n^3 + c_a n^3 = (c_m + c_a) n^3$$

elde edilir. Burada, **c_a makinedeki bir toplama işleminin süresini temsil eder.**

Sonuç olarak, çalışma süresi hesaplamalarında katsayılar (c_m, c_a) farklılık gösterebilir, ancak **büyüme düzeni değişmez.** Bu nedenle, **asemptotik analizde çarpanlar önemsiz ve algoritmanın zaman karmaşıklığı $O(n^3)$ olarak kalır.**

Yinelemeli olmayan algoritmaların analizinde sunduğumuz planın her zaman başarıyla uygulanacağını düşünmek yanıltıcı olabilir. Döngü değişkeninin düzensiz değişimi, hesaplanması karmaşık toplamalar ve özellikle ortalama durum analizinin doğasındaki zorluklar, süreci beklenenden daha karmaşık hale getirebilir. Ancak, bu sınırlamalara rağmen, önerdiğimiz yöntem birçok basit yinelemeli olmayan algoritma için oldukça etkili bir analiz çerçevesi sunmaktadır.

Şimdi, önceki örneklerden farklı bir şekilde değişen bir döngü değişkenine sahip bir algoritmayı ele alarak, analiz sürecinde karşılaşılabilecek farklı bir durumu inceleyeceğiz.

Örnek 4: Bu örnekte, pozitif bir ondalık tam sayının ikili gösteriminde kaç basamak (binary digit) içerdiğini hesaplayan bir algoritmayı ele alıyoruz. Amaç, verilen bir tam sayının binary sistemde kaç bit ile temsil edildiğini belirlemek. Bunu yapmak için sayının 2 tabanında nasıl bölündüğünü veya doğrudan logaritmik bir yaklaşım kullanarak nasıl hesaplanabileceğini inceleyeceğiz.

ALGORITHM *Binary*(n)

$count \leftarrow 1$

while $n > 1$ **do**

$count \leftarrow count + 1$

$n \leftarrow \lfloor n/2 \rfloor$

return $count$

Öncelikle, burada en sık gerçekleştirilen işlemin **while döngüsünün içindeki değil, döngünün çalışıp çalışmayacağını belirleyen $n > 1$ karşılaştırması olduğunu** belirtmeliyim. Bu karşılaştırma, döngü gövdesinin yürütülme sayısından tam olarak **bir fazla kez yapılır**, dolayısıyla analiz için hangi işlemi esas aldığımız kritik bir fark yaratmaz.

Bu örnekte daha önemli olan nokta, **döngü değişkeninin değerinin her yinelemede belirli sınırlar arasında kalması ve her adımda yaklaşık yarıya inmesidir**. Bu durum, **döngünün kaç kez çalıştığını belirlemek için alternatif bir hesaplama yöntemi kullanmamız gerektiği** anlamına gelir. Çünkü n 'nin her tekrarında yarıya düştüğünden, döngünün toplam tekrar sayısı $\log_2 n$ olarak tahmin edilebilir.

Daha kesin bir ifade ile, $n > 1$ karşılaştırmasının yapılma sayısı $\lfloor \log_2 n \rfloor + 1$ **olarak bulunur**. Bu değer, n 'nin ikili gösterimindeki bit sayısını verir ve doğrudan formül (2.1) ile doğrulanabilir. Alternatif olarak, **yinelemeli ilişkiler (recurrence relations) kullanarak** da aynı sonuca ulaşabiliriz, ancak bu teknik daha çok yinelemeli (recursive) algoritmaların analizine uygun olduğu için bir sonraki bölümde ele alınacaktır.

2. ÖZYİNELEMELİ (RECURSİVE) ALGORİTMALARIN ANALİZİ

Bu bölümde, **algoritma analizinin yinelemeli (recursive) algoritmalara nasıl uygulanacağını** ele alıyoruz. Başlangıç olarak, **yineleme kavramını yeni öğrenenler için klasik bir örnek olan faktöriyel fonksiyonunu** inceliyoruz.

Faktöriyel, $n!$ olarak tanımlanır ve şu şekilde ifade edilir:

$$n! = (n - 1)! \times n (n \geq 1)$$

Ayrıca, **temel durum (base case) olarak** $0! = 1$ tanımlanmıştır.

Bu ilişkiden yola çıkarak, $F(n) = F(n - 1) \times n$ **formülünü kullanan bir yinelemeli algoritma yazabiliriz.**

Örnek1:

ALGORITHM $F(n)$
if $n = 0$ **return** 1
else return $F(n - 1) * n$

Bu analizde, algoritmanın **girdi büyüklüğü olarak doğrudan n değerini** kullanıyorum ve ikili gösterimdeki bit sayısını dikkate almıyorum. **Temel işlem çarpma olduğu için**, bu işlemin kaç kez yürütüldüğünü $M(n)$ ile gösteriyorum.

Faktöriyel fonksiyonu **$F(n)$, $n!$**

$$F(n) = F(n - 1) \cdot n \text{ for } n > 0,$$

formülüne göre hesaplandığından, $M(n)$ değeri de bu formüle uygun olmalıdır. Bu noktada, **çarpma işlemlerinin toplam sayısını belirlemek için bir eşitlik kurulacaktır, $M(n)$ belirli bir eşitliği sağlamalıdır**

$$M(n) = \underbrace{M(n - 1)}_{\text{to compute } F(n-1)} + \underbrace{1}_{\text{to multiply } F(n-1) \text{ by } n} \text{ for } n > 0.$$

Faktöriyel hesaplamasında, $F(n-1)$ hesaplanırken $M(n-1)$ çarpma işlemi yapılır ve son olarak sonuç n ile çarpılır, yani toplamda bir ek çarpma daha gerçekleşir. Bu durum, çarpma işlemlerinin sayısını tanımlayan $M(n)$ fonksiyonunun $M(n)=M(n-1)+1$ şeklinde bir yineleme bağıntısı (recurrence relation) olarak ifade edilmesine yol açar.

Yineleme bağıntıları, **algoritma analizinde ve uygulamalı matematikte önemli bir rol oynar** ve genellikle ayrık matematik derslerinde ayrıntılı olarak incelenir. **Buradaki hedefimiz, $M(n)$ fonksiyonunu yalnızca n cinsinden açık bir formüle dönüştürmektir.**

Ancak, bu bağıntıyı çözerken **sınırsız sayıda olası çözüm dizisi elde edebiliriz.** Tek bir doğru çözümü bulmak için, **başlangıç koşulu (initial condition) gereklidir.** Algoritmanın yinelemeli çağrılarını durdurduğu noktaya baktığımızda, **$n=0$ durumunda 1 döndürüldüğünü görüyoruz.** Bu koşul, yineleme bağıntısını çözerken başlangıç değerini belirlemede kullanılacaktır.

Burada iki önemli noktaya dikkat çekmek istiyorum. **Birincisi,** algoritmanın yinelemeli çağrılarını $n = 0$ olduğunda durduğundan, bu durumda **çarpma işlemi yapılmaz** ve dolayısıyla $M(0) = 0$ olarak tanımlanmalıdır. **İkincisi,** yineleme ilişkisini tanımlarken, **$M(n) = M(n - 1) + 1$ formülü** ile her adımdaki ek çarpma işlemini ifade ettik.

Bu noktada, algoritmanın **çarpma sayısını belirleyen yineleme ilişkisini (recurrence relation) ve başlangıç koşulunu** kurmuş olduk:

$$M(n) = M(n - 1) + 1, \text{ for } n > 0$$

$$M(0) = 0$$

Bu analizde aslında **iki yinelemeli fonksiyonla** ilgileniyoruz. **İlki,** faktöriyel fonksiyonu $F(n)$, şu şekilde tanımlanıyor:

$$F(n) = F(n - 1) \cdot n, \text{ for } n > 0$$

$$F(0) = 1$$

İkinci fonksiyon ise, **faktöriyel hesaplamak için gerekli çarpma sayısını belirleyen $M(n)$ fonksiyonudur.** Bu fonksiyon, yinelemeli algoritmanın yürütülme sürecini analiz etmek için kullanılacaktır.

Yineleme bağıntısı $M(n)=M(n-1)+1$ şeklinde tanımlandığından, **geri yerine koyma yöntemi (backward substitutions)** ile çözüm arıyoruz. Bağıntıyı adım adım açarak $M(n)=M(n-i)+i$ genel formülüne ulaşıyoruz. **Başlangıç koşulu $M(0)=0$** olduğundan, **$i = n$** yerine konduğunda sonuç **$M(n)=n$** elde edilir.

Bu "bariz" sonuca ulaşmak için harcanan çabanın boşa olmadığını vurgulamak isterim. Bu yöntem, **daha karmaşık yineleme bağıntılarının çözümünde** kritik bir rol oynar. Aslında, **faktöriyel hesaplamak için yinelemeli bir yaklaşım yerine basit bir döngü kullanmak** aynı sayıda çarpma işlemi yapar ve **özyinelemenin ek yükünden kaçınır**. Yine de, bu örneği **yinelemeli algoritmaların analizini öğretmek için** ele aldık. Şimdi, bu deneyimi genelleştirerek **yinelemeli algoritmalar için sistematik bir analiz planı** oluşturacağız.

Yinelemeli Algoritmaların Zaman Karmaşıklığını Analiz Etme Adımları

1. Girdi Büyüklüğünü Belirleme:

- Algoritmanın girdi boyutunu belirleyen bir parametre seçilir.
- Örneğin, bir dizi üzerinde çalışan bir algoritma için **dizi uzunluğu n** kullanılabilir.

2. Temel İşlemi Tanımlama:

- Algoritmanın **en çok yapılan işlemi** (basic operation) belirlenir.
- Genellikle bu, **bir karşılaştırma, çarpma veya belirli bir fonksiyon çağırısı** olabilir.

3. İşlem Sayısının Girdiye Göre Değişimini İnceleme:

- Aynı büyüklükteki farklı girdilerde **temel işlemin kaç kez yürütüldüğü değişiyor mu?**
- Eğer değişiyorsa, **en iyi, ortalama ve en kötü durum analizleri** ayrı yapılmalıdır.

4. Yineleme Bağıntısı (Recurrence Relation) Kurma:

- **Algoritmanın çalışma süresi**, önceki alt problemlere bağlı olduğundan **bir yineleme bağıntısı oluşturulur**.
- Bu bağıntı, **temel işlemin kaç kez yürütüldüğünü belirleyen matematiksel bir denklem şeklindedir**.

- Ayrıca, **başlangıç koşulu (initial condition)** belirlenerek bağıntının nereden başladığı tanımlanır.

5. Bağıntıyı Çözme veya Büyüme Oranını Belirleme:

- Eğer mümkünse, yineleme bağıntısı **kapalı bir formda çözülerek** çalışma süresinin kesin bir fonksiyonu elde edilir.
- Eğer doğrudan çözilemiyorsa, en azından **büyüme düzeni (Big-O notasyonu ile)** belirlenerek zaman karmaşıklığı tahmin edilir.

Örnek 2: Hanoi Kuleleri Problemi ve Yinelemeli Çözümü

Hanoi Kuleleri (Tower of Hanoi) bulmacası, **yinelemeli algoritmaların mantığını öğretmek için kullanılan klasik bir problemdir**. Bu problemde, **farklı boyutlardaki n adet disk üç çubuktan birine sıralı olarak yerleştirilir**. Başlangıçta tüm diskler en büyükten en küçüğe sıralı şekilde ilk çubukta bulunur. Amaç, **tüm diskleri üçüncü çubuğa taşımaktır**, ancak bazı kurallar vardır:

1. Bir seferde yalnızca bir disk taşınabilir.
2. Büyük bir disk, küçük bir diskin üzerine konulamaz.
3. İkinci çubuk (yardımcı çubuk) gerektiğinde kullanılabilir.

Yinelemeli Çözüm:

Hanoi Kuleleri problemi **doğal bir yinelemeli yapıya sahiptir**.

- Eğer $n = 1$ ise, diski doğrudan **kaynak çubuktan hedef çubuğa** taşıyoruz.
- Eğer $n > 1$ ise, çözüm şu adımlardan oluşur:
 1. $n-1$ diski ilk çubuktan ikinci çubuğa taşı (üçüncü çubuğu yardımcı olarak kullan).
 2. En büyük diski doğrudan ilk çubuktan üçüncü çubuğa taşı.
 3. $n-1$ diski ikinci çubuktan üçüncü çubuğa taşı (birinci çubuğu yardımcı olarak kullan).

Bu yapı, **kendi içinde yinelemeli olarak çözülen daha küçük alt problemlere bölünmüştür**. Problemin çözümü, $T(n) = 2T(n - 1) + 1$ şeklinde bir yineleme bağıntısı oluşturur ve bu da $T(n) = 2^n - 1$ formülüne ulaşmamızı sağlar. **Bu nedenle, Hanoi Kuleleri probleminin zaman karmaşıklığı $O(2^n)$, yani üstel büyüme gösterir.**

Hanoi Kuleleri Probleminin Yinelemeli Analizi

Hanoi Kuleleri algoritmasını analiz etmek için, **disk sayısını n olarak seçiyoruz ve temel işlem olarak tek bir diskin hareket ettirilmesini kabul ediyoruz. Hareket sayısını $M(n)$ ile ifade ettiğimizde, yineleme bağıntısı şu şekilde elde edilir:**

$$M(n) = 2M(n-1) + 1, M(1) = 1$$

Bu bağıntıyı **geri yerine koyma (backward substitution) yöntemiyle** çözüyoruz:

$$M(n) = 2^i M(n-i) + 2^i - 1$$

Başlangıç koşulu $M(1) = 1$ yerine konduğunda, çözüm şu hale gelir:

$$M(n) = 2^n - 1$$

Bu sonuç, **Hanoi Kuleleri probleminin üstel ($O(2^n)$) karmaşıklığa sahip olduğunu gösterir**. Küçük n değerleri için uygulanabilir olsa da, büyük n değerlerinde **hesaplama süresi hızla artar**, bu da problemin doğasından kaynaklanan bir zorluktur.

Yinelemeli Algoritmelerde Verimsizlik ve Çağrı Ağacı Analizi

Yinelemeli algoritmaların **basit görünmesine rağmen verimsiz olabileceği** unutulmamalıdır. Bir algoritma birden fazla kez kendini çağırıyorsa, **çağrılarının yapısını anlamak için bir çağrı ağacı oluşturmak** faydalı olabilir.

Hanoi Kuleleri örneğinde, **ağaçtaki düğüm sayısı algoritmanın yaptığı toplam çağrıları temsil eder** ve şu toplam ile hesaplanır:

$$C(n) = \sum_{l=0}^{n-1} 2^l = 2^n - 1$$

Bu da, Hanoi Kuleleri algoritmasının toplam çağrı sayısının da üstel olduğunu doğrular. Bu nedenle, yinelemeli algoritmaların analizinde dikkatli olunmalı ve performanslarını anlamak için çağrı ağaçları gibi görsel araçlar kullanılmalıdır.

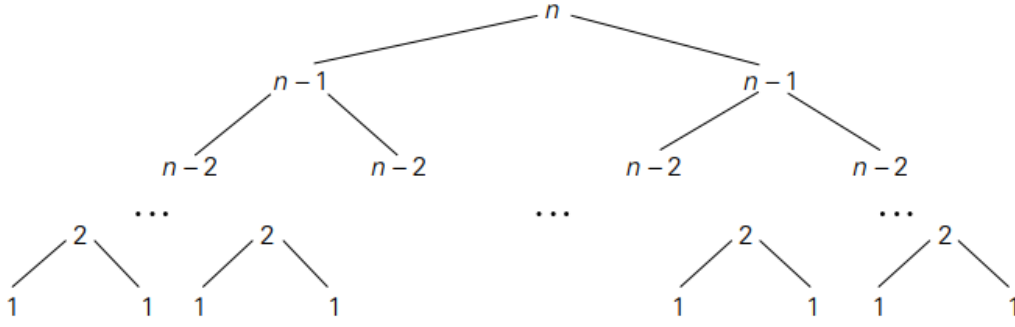


FIGURE 2.5 Tree of recursive calls made by the recursive algorithm for the Tower of Hanoi puzzle.

Sonuç olarak, iterasyon ağacındaki toplam düğüm sayısı, daha önce hesapladığımız hareket sayısı $2^n - 1$ ile uyumludur. Bu, yinelemeli algoritmanın analizinde hem matematiksel çözümün hem de çağrı ağacının aynı sonucu verdiğini göstererek, yöntemin doğruluğunu pekiştirir.

Örnek 3:

ALGORITHM *BinRec*(n)

if $n = 1$ **return** 1

else return *BinRec*($\lfloor n/2 \rfloor$) + 1

Örnek 3: Yinelemeli Bir Algoritmanın Analizi

Bu örnekte, önceki bölümde ele alınan bir algoritmanın **yinelemeli versiyonunu** analiz ediyorum. **Algoritmanın temel işlemi**, bir çağrıda yapılan **toplama işlemidir**. Toplam işlem sayısını $A(n)$ olarak ifade ediyorum. Yineleme bağıntısı şu şekildedir:

$$A(n) = A(\lfloor n/2 \rfloor) + 1, \text{ for } n > 1$$

Başlangıç koşulu ise, **algoritmanın $n=1$ durumunda toplam işlemi yapmadığı** için:

$$A(1) = 0$$

Ancak, bağıntının içinde $\lfloor \frac{n}{2} \rfloor$ ifadesinin bulunması, **geri yerine koyma (backward substitution) yöntemini** doğrudan uygulamayı zorlaştırır. Bu nedenle, **standart**

yöntem olarak $n = 2^k$ şeklindeki değerler için çözüm buluyorum ve ardından bu çözümü genelleştiriyorum.

Çözüm:

$$A(2^k) = A(2^{k-1}) + 1, \text{ for } k > 0$$

$$A(2^0) = 0$$

Geri yerine koyarak açtığımda şu kalıbı elde ediyorum:

$$A(2^k) = A(1) + k = k$$

Son olarak, $n = 2^k$ olduğundan, $k = \log_2 n$ yerine koyarak:

$$A(n) = \log_2 n \in \Theta(\log n)$$

Bu, algoritmanın **zaman karmaşıklığının logaritmik olduğunu** gösteriyor. Ayrıca, **tüm n değerleri için kesin formülün** biraz daha hassas bir ifade ile belirtilebileceğini de belirtiyorum:

$$A(n) = \lfloor \log_2 n \rfloor$$

Bu sonuç, algoritmanın **zaman karmaşıklığının büyük girişler için oldukça verimli** olduğunu gösterir.